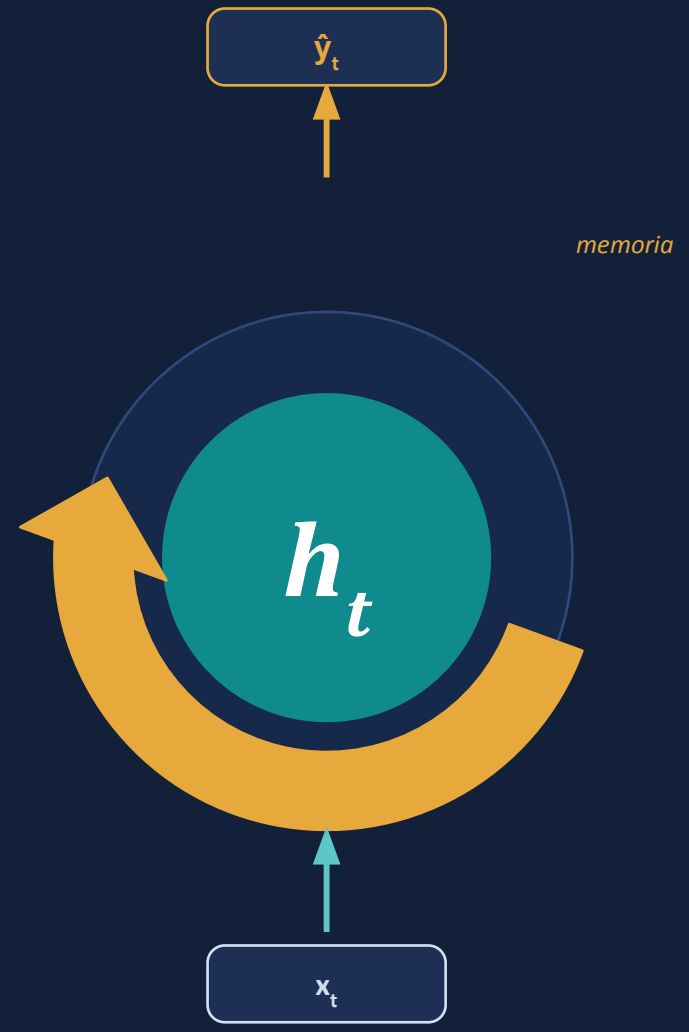
FACULTAD DE CIENCIA Y TECNOLOGÍA · LIC. EN SISTEMAS DE INFORMACIÓN

Redes Neuronales Recurrentes (RNN)

De la foto a la película: aprender cuando el orden importa

- Intuición gráfica: qué es el estado oculto
- Cómo se evalúa el error de una secuencia
- El ejemplo aplicado más simple, con números
- Guía de fórmulas de entrenamiento (BPTT)



Docentes

BioIng. Alejandro Hadad · Mg. Santiago Raul Díaz . A.S. J. López

02

¿Dónde estamos? De la MLP a la CNN, y de la CNN a la RNN

Toda arquitectura es una apuesta sobre qué estructura tienen los datos

MLP

Perceptrón multicapa

- Entrada de tamaño FIJO y sin estructura.
- Cada peso mira una posición concreta del vector.
- Si permuto las entradas, hay que reentrenar.

$$y = f(W_2\,\phi(W_1 x + b_1) + b_2)$$

Comparte: nada

Invarianza: ninguna

CNN

Red convolucional

- Un mismo filtro recorre TODA la imagen.
- Detecta el patrón esté donde esté (traslación).
- Pocos parámetros para muchísimos píxeles.

$$(x * k)[i] = \sum_u x[i+u]\,k[u]$$

Comparte: el filtro, en el ESPACIO

Invarianza: a la traslación

RNN

Red recurrente

- Una misma celda recorre TODA la secuencia.
- Arrastra un resumen del pasado: el estado h.
- Acepta secuencias de cualquier longitud.



Comparte: la celda, en el TIEMPO

Invarianza: al instante en que ocurre

La pregunta que decide la arquitectura: ¿el ORDEN de mis datos significa algo? Si la respuesta es sí, necesito memoria.

03

El problema: cuando el orden cambia el significado

Analogía — una foto es un instante; una película es una foto que recuerda las anteriores

Mismas palabras, sentido opuesto

Juan | **le prestó** | **plata** | **a** | **Pedro**

→ **Juan es el acreedor**

Pedro | **le prestó** | **plata** | **a** | **Juan**

→ **Pedro es el acreedor**

Un MLP que recibe la bolsa de palabras ve EXACTAMENTE el mismo vector en los dos casos. El orden es información, y ese modelo la tira a la basura.

Secuencias en Sistemas de Información

- Logs y trazas de auditoría
- Series de ventas y demanda
- Cotizaciones y series financieras
- Telemetría de sensores / IoT
- Texto, tickets y chatbots
- Clickstream y navegación web

La regla práctica

Si al mezclar aleatoriamente las filas de tu dataset el problema sigue teniendo sentido → MLP o CNN.

Si al mezclarlas el problema se destruye → tenés una secuencia y necesitás una RNN.

04

Intuición gráfica: una celda que se pasa notas a sí misma

Analogía — la olla y la receta

La olla y la receta

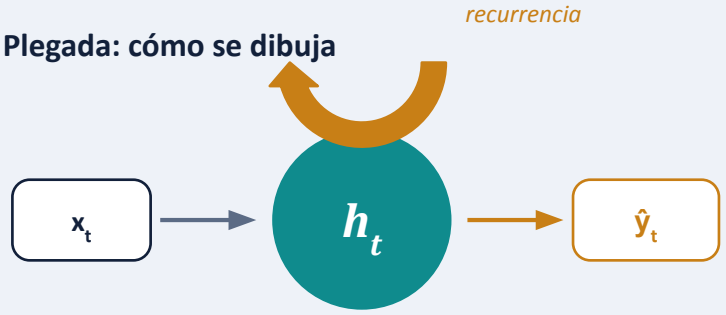
Los ingredientes (xt) entran de a uno.

La receta (los pesos W) es siempre la MISMA.

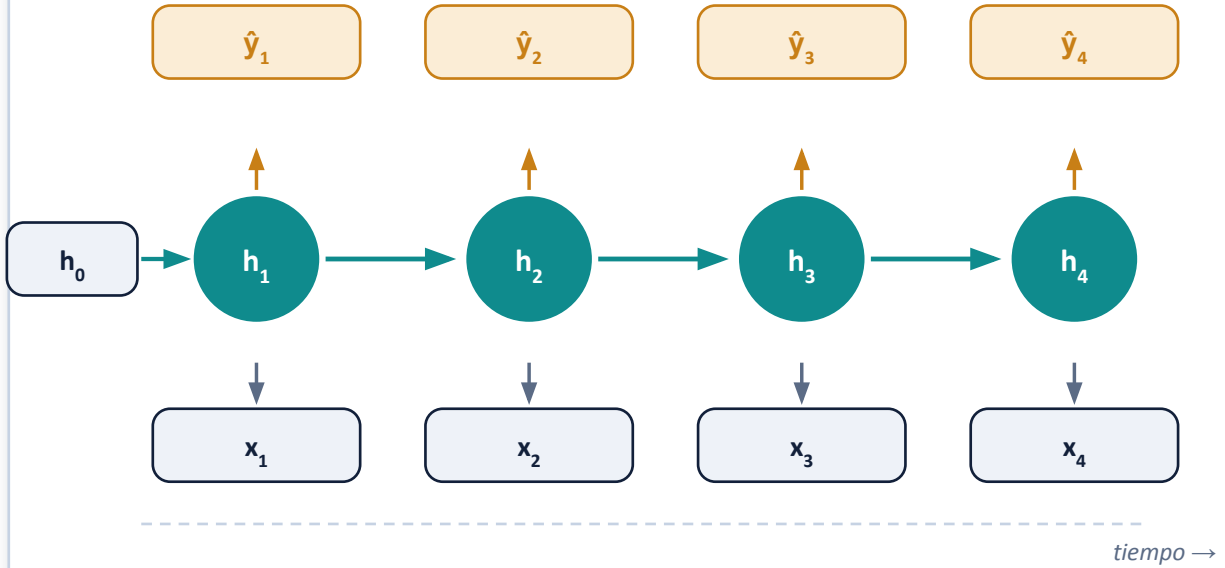
La olla (ht) va acumulando el sabor de todo lo que entró antes.

Al final probás la olla: eso es el estado oculto.

Plegada: cómo se dibuja



Desplegada en el tiempo: la misma celda, copiada en cada instante



Las cuatro flechas horizontales usan la MISMA matriz Whh.

Las cuatro entradas usan la MISMA matriz Wxh.

ht NO guarda todo el pasado: es un resumen comprimido, de tamaño fijo (m números), de todo lo visto hasta t.

05

Un solo sello, aplicado en cada instante

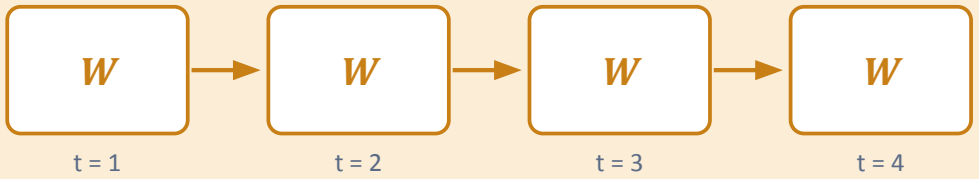
Lo que el kernel hace en el espacio, la celda recurrente lo hace en el tiempo

CNN — el filtro se desliza sobre el espacio



el mismo kernel k en cada posición

RNN — la celda se desliza sobre el tiempo



los mismos pesos W en cada instante

Consecuencia: el nº de parámetros NO crece con el largo de la secuencia.

Las tres arquitecturas, lado a lado

	MLP	CNN	RNN
Entrada	vector fijo	grilla fija	secuencia de largo variable
Comparte	nada	filtro (espacio)	celda (tiempo)
Supuesto	sin estructura	vecindad local	el pasado explica el futuro
Memoria	no tiene	campo receptivo	estado oculto ht
Parámetros	crecen con d	fijos por filtro	fijos: no dependen de T
Profundidad	nº de capas	nº de capas	nº de capas × T pasos

Ojo con la última fila: desplegar una RNN sobre T = 100 pasos equivale a una red de 100 capas con los pesos atados. De ahí saldrá, en la diapositiva 13, todo el problema del gradiente.

06

Anatomía de la celda simple (Elman, 1990)

Dos líneas de álgebra: una para la memoria, otra para la salida

1 · Actualizar la memoria

$$h_t = \tanh\left(W_{xh}\, x_t + W_{hh}\, h_{t-1} + b_h\right)$$

pasado + presente → nueva memoria

2 · Leer la memoria y decidir

$$\hat{y}_t = \mathrm{softmax}(W_{hy}\, h_t + b_y)$$

memoria → probabilidades sobre las k clases

Dimensiones y conteo de parámetros

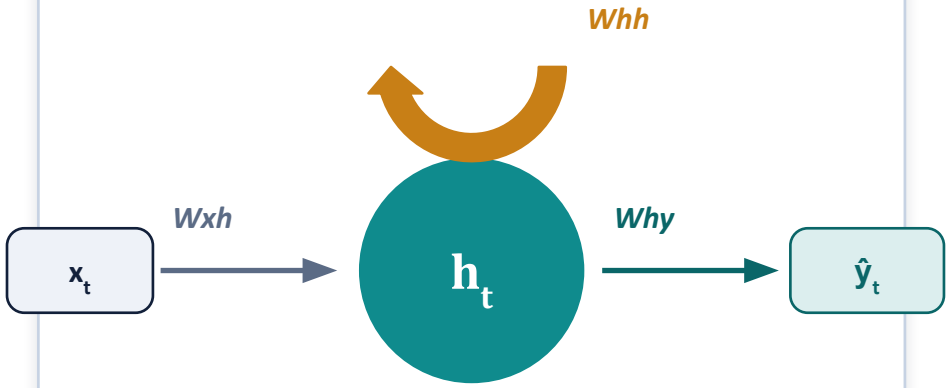
$$x_t \in \mathbb{R}^d \quad h_t \in \mathbb{R}^m \quad W_{xh} \in \mathbb{R}^{m \times d} \quad W_{hh} \in \mathbb{R}^{m \times m} \quad W_{hy} \in \mathbb{R}^{k \times m}$$

$$\#\theta = md + m^2 + km + m + k \implies \text{no depende de } T$$

$$h_0 = 0$$

d = dim. de entrada · m = tamaño del estado · k = nº de salidas · T = largo de la secuencia

Las tres matrices



Wxh: qué mirar del presente

Whh: qué conservar del pasado

Why: cómo responder

- tanh mantiene ht acotado en (−1, 1): evita que la memoria se dispare.
- softmax convierte puntajes en una distribución de probabilidad.
- h0 = 0 por convención: arrancamos sin memoria.

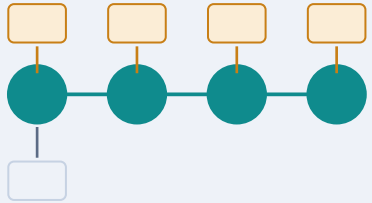
Fuente: Elman, J. L. (1990). Finding structure in time. Cognitive Science, 14(2), 179–211.

07

Cuatro formas de conectar entrada y salida

La misma celda; lo que cambia es dónde entra el dato y dónde se lee la respuesta

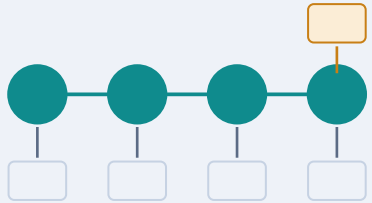
Uno a muchos



Una entrada, una secuencia de salida.

Generar la descripción de una imagen; componer una melodía.

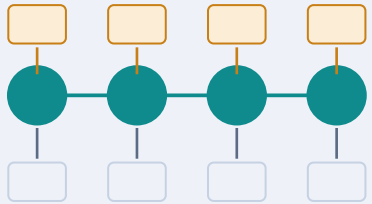
Muchos a uno



Toda la secuencia, una sola respuesta al final.

¿Esta traza de logs termina en falla? Análisis de sentimiento de un ticket.

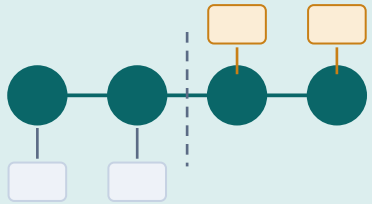
Muchos a muchos (sincronizado)



Una salida por cada instante de entrada.

Detección de anomalías en línea; etiquetar cada palabra de una oración.

Codificador y decodificador



Leo todo, resumo, y recién ahí genero.

Traducción automática; resumen de texto (Sutskever et al., 2014).

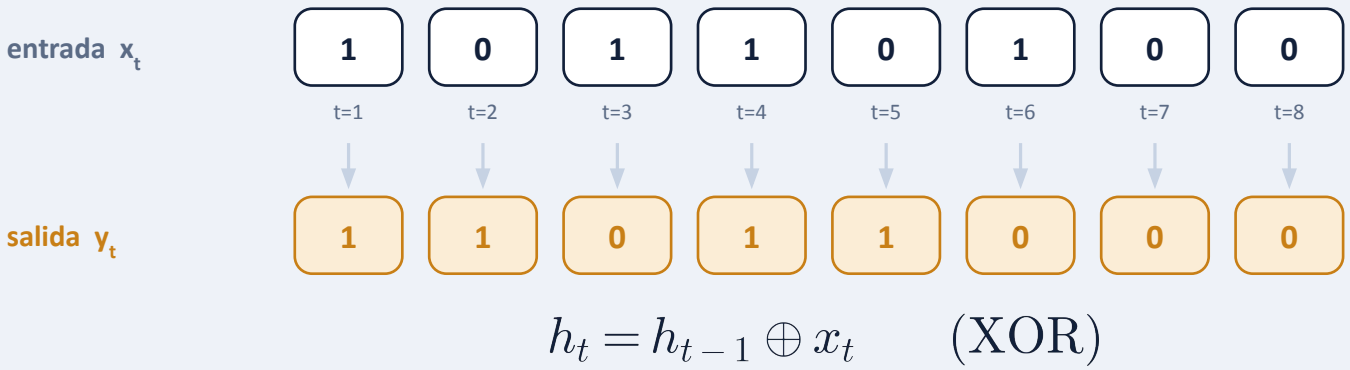
Las cuatro comparten exactamente las mismas ecuaciones de la diapositiva 6. Sólo cambia sobre qué instantes se calcula la pérdida.

Fuente: Sutskever, Vinyals y Le (2014); Goodfellow et al. (2016), §10.2–10.4.

08

El ejemplo mínimo: ¿por qué no alcanza con un MLP?

Tarea de paridad — responder 1 si la cantidad de unos vistos hasta ahora es impar



$$h_t = h_{t-1} \oplus x_t \qquad \text{(XOR)}$$

La RNN que lo resuelve

1 unidad de estado.

3 pesos y 1 sesgo.

Funciona con secuencias de 8 bits, de 80 o de 8000.

El estado h guarda un único bit: la paridad acumulada.

Intento con MLP (ventana fija)

- Hay que elegir un tamaño de ventana ANTES de ver los datos.
- Ventana de 8 bits → 8 entradas. Ventana de 800 → 800 entradas y 100× más parámetros.
- Si mañana llega una secuencia más larga, el modelo no sirve: hay que rediseñarlo.
- Nunca aprende la regla; aprende una tabla de casos.

Con RNN

- No hay que elegir ninguna ventana: la longitud es libre.
- El nº de parámetros es constante, mida lo que mida la secuencia.
- Aprende la REGLA (una recurrencia), no una tabla.
- Generaliza a largos que nunca vio en el entrenamiento.

La paridad es el ejemplo clásico de dependencia de largo alcance: cada bit importa, sin importar cuán atrás quedó.

09

El ejemplo aplicado más simple: predecir el próximo carácter

Es el autocompletado del teclado del celular, reducido a cuatro letras

Planteo del problema

Corpus: la palabra “hola”

Vocabulario: V = { h, o, l, a } ⟶ 4 símbolos

Entrada: h o l Objetivo: o l a

En cada paso la red responde una única pregunta: dado todo lo que leí, ¿cuál es la letra siguiente?

Hiperparámetros: d = 4 · m = 2 · k = 4 ⟶ 26 parámetros en total

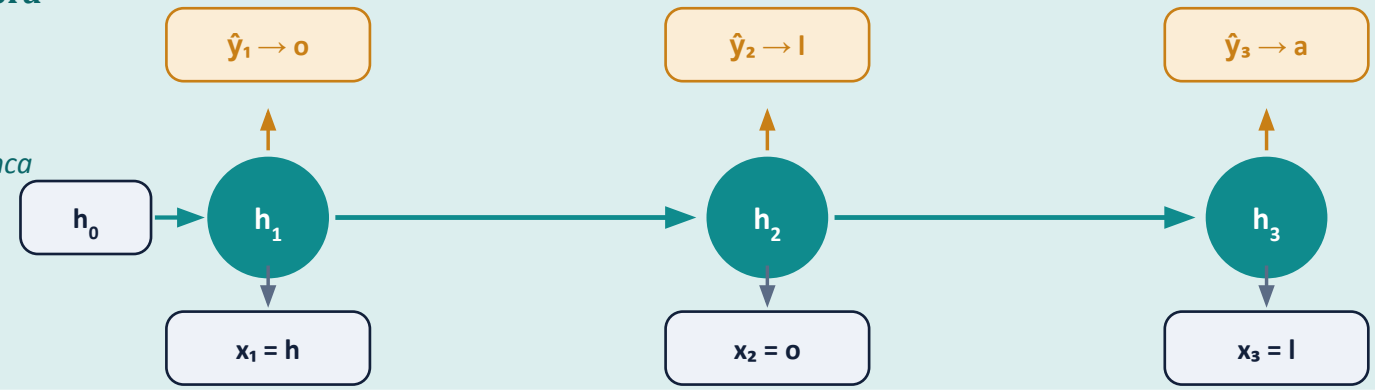
Codificación one-hot: de símbolo a vector

h	o	l	a
1 0 0 0	0 1 0 0	0 0 1 0	0 0 0 1

Igual que en la CNN, la red no entiende de letras: entiende de vectores. One-hot es la traducción más simple, sin suponer que ninguna letra se parece a otra.

La red desplegada sobre la palabra

Teacher forcing: en el entrenamiento, la entrada del paso t+1 es la letra REAL, nunca la que la red predijo.



Fuente: Karpathy, A. (2015), char-RNN; Graves, A. (2013), arXiv:1308.0850.

10

Paso a paso, con números reales

Pesos iniciales aleatorios (σ = 0,6), sesgos en cero, $h_0 = 0$ — vocabulario ordenado h, o, l, a

t	xt	at = Wxh xt + Whh ht−1	ht = tanh(at)	ŷt (probabilidades)	predice	objetivo	Lt
1	h	(0,000 ; −0,270)	(0,000 ; −0,264)	0,266 0,206 0,247 0,281	a	o	1,582
2	o	(0,278 ; −0,645)	(0,271 ; −0,569)	0,328 0,184 0,220 0,267	h	l	1,514
3	l	(−0,031 ; −0,001)	(−0,031 ; −0,001)	0,246 0,246 0,253 0,255	a	a	1,366

$$a_1 = W_{xh} x_1 + W_{hh} h_0 \;\Rightarrow\; h_1 = \tanh(a_1)$$

La secuencia se recorre una sola vez, de izquierda a derecha, arrastrando ht.

Antes de entrenar

L = (1,582 + 1,514 + 1,366) / 3 = 1,487

PPL = e^1,487 = 4,42

Con V = 4 letras, tirar una moneda da PPL = 4. La red sin entrenar no sabe nada: acierta 1 de 3, y por casualidad.

Después de 300 pasos de BPTT

L = 0,002 PPL = 1,002

p(o|h) = p(l|ho) = p(a|hol) = 0,998

Los mismos 26 parámetros, movidos por el gradiente. PPL ≈ 1 significa “no me queda ninguna duda”.

Valores obtenidos ejecutando la RNN de 2 unidades en NumPy: forward exacto y BPTT con Adagrad (η = 0,5).

11

¿Cómo se evalúa el error de una secuencia?

Hay una pérdida por cada instante; el modelo se entrena con la suma de todas

Un error por paso, un total por secuencia

$$L_t = -\sum_{k=1}^{V} y_{t,k} \log \hat{y}_{t,k}$$

clasificación: entropía cruzada

$$L = \frac{1}{T}\sum_{t=1}^{T} L_t$$

pérdida total de la secuencia

$$L = \frac{1}{T}\sum_{t=1}^{T} (y_t - \hat{y}_t)^2$$

regresión: error cuadrático medio

$$\mathrm{PPL} = e^{L}$$

perplejidad: ¿entre cuántas opciones dudo?

El error se acumula en cadena

Si la red se equivoca en t = 3, ese error queda escrito en h_3, y h_3 contamina h_4, h_5, h_6…

En una RNN no hay errores independientes: hay errores que se propagan hacia adelante en el tiempo.

Entrenamiento vs. producción: el detalle que casi siempre se pasa por alto

Teacher forcing — al entrenar

En cada paso le damos como entrada el valor REAL del paso anterior. Converge mucho más rápido y estabiliza el gradiente.

Free running — en producción

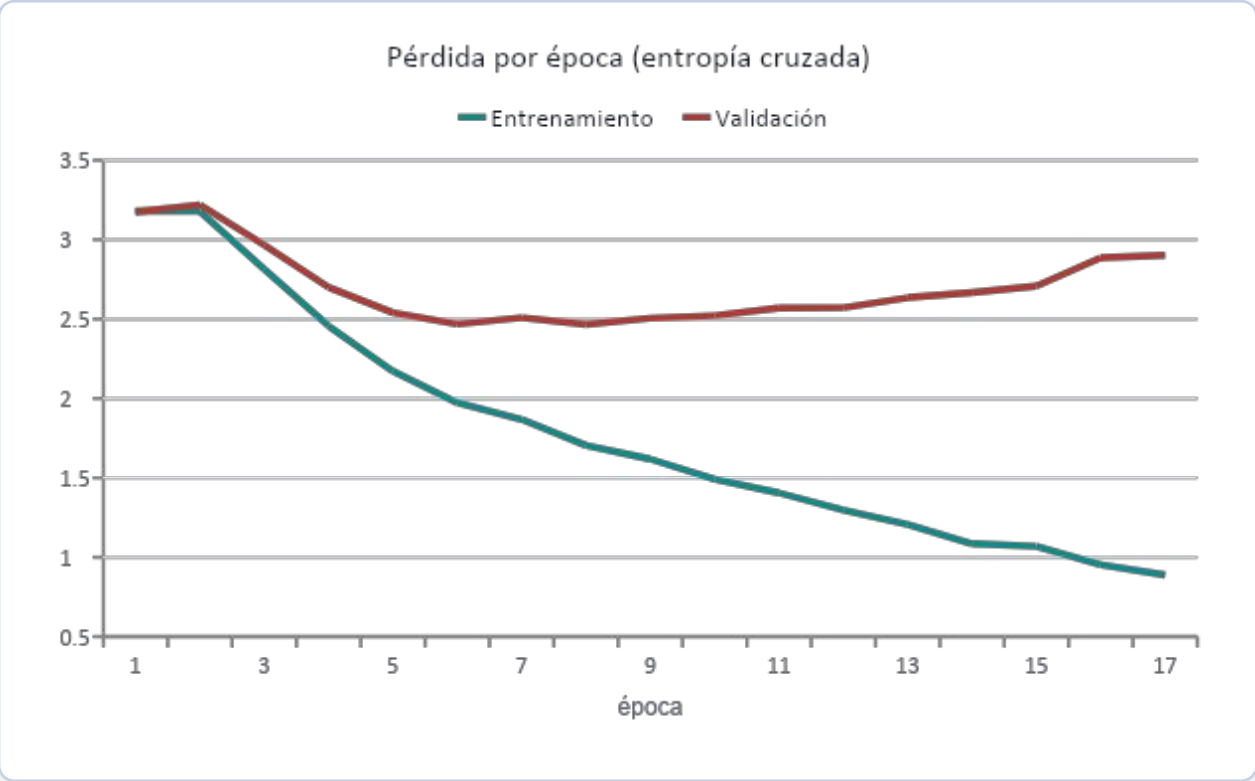
La red se alimenta de su propia predicción. Un error temprano se realimenta y la salida se desvía cada vez más: exposure bias. Hay que medir el error TAMBIÉN en este modo.

Fuente: Goodfellow et al. (2016), §10.2.1; Graves (2013).

12

Evaluar de verdad: no te mientas con la métrica

Curvas reales del char-RNN de la diapositiva 9, entrenado sobre un texto de 454 caracteres



Lo que dicen los números

```
época   52 →  val 2,455   (PPL 11,6)  ← mínimo
época 160 →  train 0,89   (PPL  2,4)
                          val 2,90   (PPL 18,2)
```

A partir de la época ~52 la red deja de aprender y empieza a memorizar.

Cuatro reglas para series y secuencias

- Partición TEMPORAL: entreno con el pasado, valido con el futuro. Mezclar al azar es fuga de información.
- Validación walk-forward: mover el origen y reevaluar.
- Línea de base de persistencia: $\hat{y}_{t+1} = y_t$. Si no le ganás, no aprendiste nada.
- Early stopping en el mínimo de la curva de validación.

Métricas según la tarea: clasificación por paso → exactitud, F1, matriz de confusión, perplejidad · regresión → RMSE, MAE, MAPE · secuencia completa → distancia de edición, BLEU

Fuente: Hyndman y Athanasopoulos (2021), cap. 5; Chollet (2021), cap. 10.

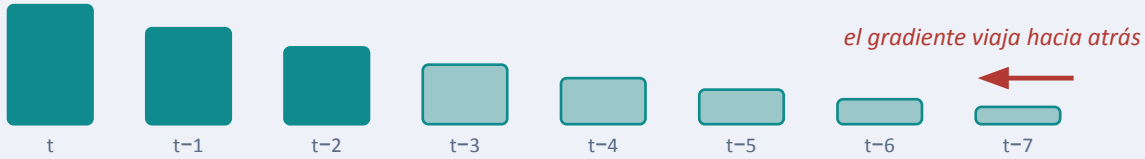
13

Entrenar = desplegar y retropropagar en el tiempo (BPTT)

Desplegar T pasos equivale a una red de T capas con TODOS los pesos atados

Analogía: el teléfono descompuesto

El error nace al final y viaja hacia atrás en el tiempo. En cada salto lo multiplica la misma matriz. Si esa matriz achica, el mensaje llega en un susurro; si agranda, llega como un grito.



Por qué pasa: el producto de jacobianos

$$\frac{\partial h_j}{\partial h_{j-1}} = W_{hh}^{\top}\,\mathrm{diag}\left(1 - h_j^2\right)$$

$$\left\| \prod_{j=k+1}^{t} \frac{\partial h_j}{\partial h_{j-1}} \right\| \le \left(\gamma \left\| W_{hh} \right\|\right)^{t-k}$$

γ = cota de |tanh'| = 1. El factor se eleva a la distancia temporal t − k.

Se desvanece · γ‖Whh‖ < 1

El gradiente decae exponencialmente. La red entrena bien lo reciente y no aprende NADA de lo lejano: olvida. Es el caso habitual.

Explota · γ‖Whh‖ > 1

El gradiente crece exponencialmente: saltos enormes, pérdida en NaN, entrenamiento inestable. Se detecta mirando la norma del gradiente.

Remedios: recorte de gradiente (clipping) · BPTT truncado a k pasos · inicialización ortogonal de Whh · normalización · y sobre todo: compuertas — LSTM (1997) y GRU (2014), diapositiva 14

Fuente: Werbos (1990); Bengio, Simard y Frasconi (1994); Pascanu, Mikolov y Bengio (2013).

14

Guía de fórmulas de entrenamiento

Hoja de ruta completa: de la propagación hacia adelante a la actualización de los pesos

BPTT en seis pasos

1 Hacia adelante, para t = 1…T (h₀ = 0)

$$h_t = \tanh\left(W_{xh}\, x_t + W_{hh}\, h_{t-1} + b_h\right)$$
$$\hat{y}_t = \mathrm{softmax}(W_{hy}\, h_t + b_y)$$

2 Pérdida por paso y pérdida total

$$L_t = -\sum_{k=1}^{V} y_{t,k} \log \hat{y}_{t,k} \qquad L = \frac{1}{T}\sum_{t=1}^{T} L_t$$

3 Error en la capa de salida y su gradiente

$$\delta_t^y = \hat{y}_t - y_t \qquad \frac{\partial L}{\partial W_{hy}} = \sum_{t=1}^{T} (\hat{y}_t - y_t) h_t^{\top}$$

4 Retropropagar en el tiempo (t = T…1)

$$\delta_t^h = \left(W_{hy}^{\top}\delta_t^y + W_{hh}^{\top}\delta_{t+1}^a\right), \qquad \delta_t^a = \left(1 - h_t^2\right) \odot \delta_t^h$$

5 Acumular los gradientes de los pesos

$$\frac{\partial L}{\partial W_{xh}} = \sum_{t=1}^{T} \delta_t^a x_t^{\top} \qquad \frac{\partial L}{\partial W_{hh}} = \sum_{t=1}^{T} \delta_t^a h_{t-1}^{\top}$$

6 Recortar y actualizar

$$\text{si } \|g\| > \theta: \quad g \leftarrow \frac{\theta}{\|g\|}\, g \qquad \theta \leftarrow \theta - \eta \nabla_\theta L$$

LSTM — Hochreiter y Schmidhuber (1997)

olvidar · guardar · mostrar

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$
$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$
$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$
$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$
$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$
$$h_t = o_t \odot \tanh(c_t)$$

GRU — Cho et al. (2014)

dos compuertas en vez de tres; menos parámetros

$$z_t = \sigma(W_z[h_{t-1}, x_t]) \qquad r_t = \sigma(W_r[h_{t-1}, x_t])$$
$$\tilde{h}_t = \tanh(W[r_t \odot h_{t-1}, x_t])$$
$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

σ = sigmoide (compuerta en [0,1]) · ⊙ = producto elemento a elemento · [a, b] = concatenación · los gradientes de los T pasos se SUMAN antes de actualizar

15

Para llevarse, y dónde seguir leyendo

Tres ideas y catorce referencias

El estado oculto

Una RNN es un MLP con una sola línea extra: la que le pasa su propia memoria del instante anterior. Todo lo demás sale de ahí.

Compartir en el tiempo

Los mismos pesos en cada instante, igual que el kernel de la CNN en cada posición. Por eso acepta secuencias de cualquier largo.

El precio: el gradiente

Desplegar en el tiempo multiplica jacobianos. De ahí el desvanecimiento, el clipping, el BPTT truncado y las compuertas.

Bibliografía

- Rumelhart, D. E., Hinton, G. E. y Williams, R. J. (1986). Learning representations by back-propagating errors. Nature, 323, 533–536.
- Elman, J. L. (1990). Finding structure in time. Cognitive Science, 14(2), 179–211.
- Werbos, P. J. (1990). Backpropagation through time: what it does and how to do it. Proceedings of the IEEE, 78(10), 1550–1560.
- Bengio, Y., Simard, P. y Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. IEEE Trans. on Neural Networks, 5(2), 157–166.
- Hochreiter, S. y Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735–1780.
- Pascanu, R., Mikolov, T. y Bengio, Y. (2013). On the difficulty of training recurrent neural networks. ICML, 1310–1318.
- Graves, A. (2013). Generating sequences with recurrent neural networks. arXiv:1308.0850.
- Cho, K. et al. (2014). Learning phrase representations using RNN encoder–decoder for statistical machine translation. EMNLP, 1724–1734.
- Sutskever, I., Vinyals, O. y Le, Q. V. (2014). Sequence to sequence learning with neural networks. NeurIPS 27, 3104–3112.
- Goodfellow, I., Bengio, Y. y Courville, A. (2016). Deep Learning, cap. 10. MIT Press.
- Vaswani, A. et al. (2017). Attention is all you need. NeurIPS 30, 5998–6008.
- Hyndman, R. J. y Athanasopoulos, G. (2021). Forecasting: Principles and Practice (3ª ed.). OTexts.
- Chollet, F. (2021). Deep Learning with Python (2ª ed.), cap. 10. Manning.
- Zhang, A., Lipton, Z. C., Li, M. y Smola, A. J. (2023). Dive into Deep Learning, cap. 9–10. Cambridge University Press.